Rishita Yadav, 2025, 13:4
ISSN (Online): 2348-4098
ISSN (Print): 2395-4752

International Journal of Science, Engineering and Technology

An Open Access Journal

Optimizing CUDA Kernels for High-Performance Canny Edge Detection

[1]Rishita Yadav, [2]Harshul Gupta
Columbia University Department of applied maths

Abstract - We present a comprehensive optimization of the Canny Edge Detection algorithm using NVIDIA CUDA to achieve high-performance GPU acceleration. Our approach integrates warp-level execution, shared memory tiling, memory coalescing, and page-locked transfers to systematically enhance computational efficiency. Evaluated on a subset of the ILSVRC 2017 dataset across six image resolutions (128×128 to 1024×1024), the optimized kernels achieve up to 56× speedup over a baseline OpenCV implementation, with the shared memory version reaching 83.3% compute and memory throughput. Profiling with NVIDIA Nsight Compute reveals that memory transfer bottlenecks remain the primary limitation, but asynchronous transfers and stream concurrency further improve performance. These results demonstrate the potential of advanced CUDA optimization techniques for enabling real-time, high-resolution edge detection in large-scale image processing applications.

Keywords - CUDA, GPU optimization, Canny Edge Detection, parallel computing, image processing

I. INTRODUCTION

Edge detection is a fundamental task in computer vision, serving as the foundation for object recognition, image segmentation, and motion analysis. Among various edge detection algorithms, the Canny edge detector(1) remains one of the most widely used due to its effectiveness in reducing noise and accurately detecting edges. However, the computational cost of Canny edge detection becomes a significant bottleneck when processing large-scale datasets such as ImageNet(2), especially in real-time applications or on high-resolution images.

To address this computational challenge, this work explores the acceleration of the Canny edge detection pipeline using NVIDIA CUDA, a parallel computing platform and programming model for GPUs(3). The CUDA-based implementation of Canny edge detection leverages GPU-optimized techniques such as shared memory, efficient memory access patterns, and CUDA streams to exploit the massive parallelism inherent in modern GPUs. We compare the performance of our CUDA-based approach with an OpenCV-based implementation(4) to highlight the advantages of GPU acceleration.

The experimental evaluation is conducted on a subset of the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2017 dataset (2). To ensure a comprehensive analysis, the images were pre-processed and resized to six different resolutions: 128 × 128, 256 × 256, 320 × 320, 512 × 512, 640 × 640, and 1024 × 1024. The performance of both methods is evaluated in terms of execution time and accuracy. Our results demonstrate that the CUDA-based implementation achieves significant speedups compared to the OpenCV implementation, particularly at higher resolutions where GPU parallelism is more effectively utilized. This study underscores the potential of GPU-accelerated edge detection for real-time applications and large-scale image processing tasks.

The contributions of this paper are as follows:

- We propose a GPU-accelerated implementation of the Canny edge detection pipeline using CUDA.
- We compare the CUDA implementation with the OpenCV based approach on various image resolutions.
- We evaluate the performance of both methods using the ILSVRC 2017 dataset (2) to showcase scalability and computational efficiency.

The rest of this paper is organized as follows:
Section

II reviews related work, Section III describes the proposed GPU-accelerated Canny edge detection pipeline, Section IV presents the experimental setup and results, and Section V concludes the paper with future directions.

Figure 1: Comparison between outputs of CUDA and OpenCL outputs

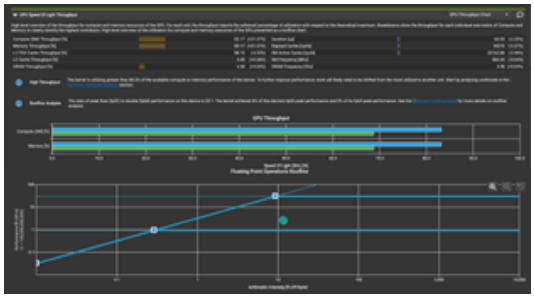

Figure 2: Outputs from different steps of processing

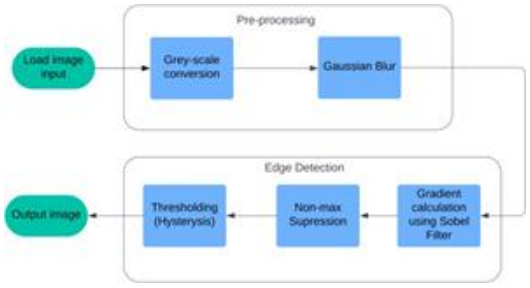

Figure 3: Block diagram of the methodology pipeline

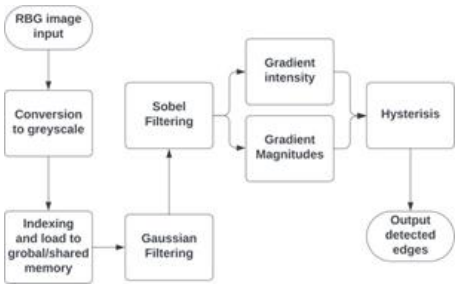

Figure 4: Flowchart of the implementation steps

Background and Related work

Numerous researchers have explored the parallel implementation of the Canny edge detection algorithm, employing diverse hardware and optimization techniques. SHI Weizhong et al.(5) developed an FPGA-based optimization algorithm, enhancing processing speed for 512 × 512 gray-scale images. Jin et al.(6) utilized the ZC706 platform and SDSoC environment to accelerate edge detection, achieving a 16.97× speedup for the same resolution. Similarly, Keqiang et al.(7) optimized the Canny operator on the TI DSP TMS320C6678 processor, improving performance at an 800 × 600 resolution. Xiangjiao et al.(8) implemented a parallel Canny algorithm using TBB and C++, achieving a 3.673× acceleration on 22.89M gray-scale images using a quad-core CPU. Yue et al.(9) executed the algorithm on GPUs with OpenGL, ensuring real-time performance for 256 × 256 images. Bin et al.(10) introduced a GPU+CPU-based approach, realizing a 5.39× speedup for 1024 × 1024 images.

Further, researchers like Jin et al.(11) proposed a Canny algorithm under the OpenCL architecture, reporting a 1.42×speedup for 2048 × 1536 images without factoring in data transfer. Mochurad(12) leveraged CUDA to achieve a 68× performance boost for 10240 × 10240 gray-scale images, while Horvath et al.(13) reported a 101× speedup for 1280 × 720 images on CUDA. Others explored batch processing improvements via Hadoop clusters(14) (15). Enhanced algorithms for specific applications have also been proposed; for instance, FPGA implementations optimized for real-time 512 × 512 edge detection(16) (17) and FPGA-based methods for mobile vision systems. Suwen et al.(18) enhanced weak edge detection capabilities on FPGA, while Shengxiao et al.(19) achieved a 64× acceleration for 512 × 512 images on GPUs. Advanced applications include Fuqiang et al.'s(20) low-error line segment detection using FPGA-based Canny algorithms, Sivakumar and Janakiraman's(21) MRI ROI segmentation on FPGA, and Hongye's(22) fingerprint acquisition system optimized with DSP. Other examples include systems integrating DSP and FPGA for verticality recognition, CUDA-based

Gaussian mixture models for real-time video analysis, and satellite-based ship tracking using multi-feature Canny algorithms on embedded GPUs.

In summary, three main research avenues are evident: parallelizing the Canny operator on various architectures (CPU, FPGA, DSP, GPU, and Hadoop clusters), enhancing the operator's computational efficiency, and applying it to practical systems for acceleration. While FPGA and CUDA exhibit high acceleration potential, they face challenges like cost, debugging complexity, and limited portability. Most existing studies focus on a single parallel technology, lacking comparative analysis across different models. Considering the heterogeneity of modern computing platforms, efficiently utilizing diverse processors, such as CPUs and GPUs, is critical.

This paper addresses these gaps by exploiting GPU storage through PyCUDA and PyOpenCL for high-speed Canny edge detection. The study analyzes and optimizes GPU memory access modes (global, local, and constant) for parallelized Gaussian filtering and gradient computation. Enhancements include optimizing template operations and extending images to align with GPU architectures, reducing storage demands, and improving vectorized computation. Experimental results validate the effectiveness of these optimizations in achieving efficient parallel execution and high-speed edge detection.

II. METHODOLOGY

The implementation of the Canny Edge Detection algorithm was carried out through four different approaches, namely:

- Naive Implementation
- Shared Memory Optimization
- Memory Coalescing
- Warp-Level Optimization

Each implementation followed a structured pipeline consisting of preprocessing, edge detection, and optimization steps. The key phases of the methodology are described below:

Preprocessing Steps

Before applying the edge detection algorithm, the input image underwent the following preprocessing steps to prepare it for gradient calculations:

Grayscale Conversion

The input color image was first converted to a grayscale image to reduce computational complexity. This was achieved using a weighted sum of the RGB channels, as per the standard formula:

$$Igray = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$$

where are the red, green, and blue pixel intensities, respectively, as can be seen from figure 2.

Gaussian Blurring

To reduce noise and smooth the image, a Gaussian blur was applied using a convolution filter. A 2D Gaussian kernel was used, defined as:
where is the standard deviation of the Gaussian distribution. The kernel size and were chosen to balance between noise reduction and edge preservation.

Edge Detection Steps

The Canny Edge Detection algorithm was implemented through the following stages:

Gradient Calculation (Sobel Filtering)

The Sobel operator was used to calculate the gradients of the smoothed image in the x and y directions. The Sobel filters are defined as

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The gradients and were convolved with the image to compute the gradient magnitude and direction :

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Non-Maximum Suppression

To thin the edges and retain only the strongest ones, non-maximum suppression was applied. For each pixel, the gradient magnitude was compared to its neighbors along the direction of the gradient. If the

pixel value was not the local maximum, it was suppressed (set to zero).

Thresholding (Hysteresis)

To finalize the edges, a double-thresholding technique was applied:

- High threshold: Retain strong edge pixels.
- Low threshold: Include weak edge pixels if they are connected to strong edges. This step ensured the detection of meaningful edges while reducing noise- induced artifacts.

CUDA Implementation Optimization Strategies

The four implementations were designed to progressively optimize the performance of the Canny Edge Detection algorithm on GPU hardware. The optimizations are summarized as follows:

Naive Implementation

In the naive implementation, each pixel computation was handled independently without any optimization. The image data was read directly from global memory, and computations were performed sequentially on the GPU.

Tiling and Warp Optimizations

To further optimize the algorithm, warp-level operations were incorporated to reduce thread divergence and improve parallel execution efficiency. The following strategies were employed:

- Warp intrinsics: Built-in functions were used to share data within a warp, minimizing shared memory usage for certain calculations.
- Gradient comparison: Non-maximum suppression was optimized using warp-level synchronization to process neighboring pixels efficiently.

Shared/Private Memory Optimizations

To reduce global memory access latency, shared memory was utilized to load image tiles into faster on-chip memory. Threads within a block cooperatively loaded a portion of the image into shared memory, allowing for faster access during convolution and gradient calculations.

- Tile-based processing: Image data was divided into small tiles that fit into shared memory.
- Boundary handling: Extra rows and columns were
- loaded to handle boundary pixels during filtering.

Kernel Configurations

The kernel configurations for each implementation were carefully selected to maximize GPU utilization. A block size of threads was used for most implementations, as it strikes a balance between parallelism and shared memory usage. The grid size was dynamically calculated based on the input image dimensions to ensure full coverage of the image. For warp-level optimization, the block size was aligned to warp boundaries (multiples of 32) to take advantage of warp-level intrinsics and minimize thread divergence. These configurations were experimentally validated to achieve optimal performance across all implementations.

OpenCL implementation

The same methodology was implemented in OpenCL as well. The output of both methods were compared. The edge detection results of both can be seen in figure 1.

Flowchart and Pseudocode

The implemented algorithm focuses on edge detection through a multi-stage pipeline consisting of Pre-processing and Edge Detection, as illustrated in Figures 3 and 4. The input RGB image is first converted to greyscale to reduce computational complexity while retaining critical edge information. Gaussian filtering is then applied to smooth the image and minimize noise, ensuring more robust edge detection.

In the Edge Detection phase, the Sobel filter calculates gradient magnitudes and intensities to identify significant intensity changes corresponding to edges. Non-maximum suppression is applied to eliminate non-local maxima, refining the gradient response. Finally, double-thresholding with hysteresis links weak edges to strong edges, ensuring continuity while discarding false positives. The resulting output is the detected edges in the input image.

The implementation efficiently manages data flow by loading greyscale image data into global/shared memory, followed by sequential application of Gaussian filtering, Sobel filtering, and edge refinement techniques. This structured pipeline

optimizes memory access and computational load while achieving accurate edge detection.

III. EXPERIMENTAL SETUP

Dataset

In this work, a subset of the ILSVRC2017 dataset, consisting of 80 images, was utilized for experimental purposes. These images were preprocessed to generate a structured dataset with multiple image resolutions, enabling evaluation across varying scales. The preprocessing pipeline involved systematic cropping and resizing of images to six predefined resolutions

The computational platform

The computational platform is a virtual machine running on Google Cloud, equipped with an NVIDIA Tesla T4 GPU with 15.36 GB of memory and CUDA version 12.6. The system utilizes an Intel Xeon(R) CPU E7-2300 at 2.30 GHz with 2 cores and 6 threads. The platform operates on an x86 64 architecture and supports 64-bit processing. Memory usage for the running Python process is 98 MiB. The system is optimized for heterogeneous computing tasks, leveraging both CPU and GPU capabilities for high-performance parallel computations.

IV. RESULTS AND DISCUSSION

CUDA vs OpenCV Execution Time Comparison

This study compares the performance of different CUDA implementations (Naive, Warp Optimized, Memory Lock, Shared Memory) with OpenCV across six image resolutions: 128 × 128, 256 × 256, 320 × 320, 512 × 512, 640 × 640, and 1024 × 1024.

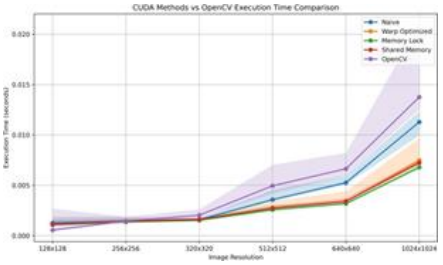

Figure 5: Execution time comparison of CUDA methods vs OpenCV for various image resolutions.

At lower resolutions (128×128 and 256×256), all methods, including CUDA and OpenCV, show similar execution times due to the minimal computational load. However, as the resolution increases beyond 512 × 512, performance differences become evident. The OpenCV implementation exhibits a steep rise in execution time, particularly at 1024×1024, indicating a lack of GPU optimizations. In contrast, CUDA-based methods scale better with increasing image size. Among the CUDA implementations, the Naive method shows moderate improvement but remains less efficient compared to the optimized approaches. The Warp Optimized and Memory Lock methods achieve the best performance, demonstrating the benefits of warp-level parallelism and memory access optimizations. The Shared Memory implementation also improves performance by reducing global memory access latency through data reuse, though it is slightly less efficient than the Warp Optimized and Memory Lock methods. At the largest resolution (1024 × 1024), the execution time for OpenCV is significantly higher than all CUDA methods, further underscoring the advantages of GPU-based parallelism and memory optimizations employed in CUDA implementations.

Shared Memory Version Analysis

The shared memory version of the gaussianBlur kernel demonstrates improved performance compared to the baseline (gaussianBlur naive) by utilizing tiled memory access and reducing redundant global memory reads. Table 1 summarizes the key results from the profiling data for GPU activities and API calls. The profiling results reveal that the gaussianBlur kernel accounts for 30.32% of the total GPU execution time, with an average runtime of 31.11 μs per call. However, Host-to- Device and Device-to-Host memory copies together contribute to over 60% of GPU activity time, indicating significant overhead. This suggests opportunities for optimization, such as using pinned memory or implementing asynchronous memory copies (3). The analysis of API calls shows that the most time-consuming operations are cuCtxCreate (64.21%) and cuCtxDetach (25.50%), which are associated with context initialization and cleanup. These calls are one-time costs and do not directly impact kernel execution performance. In contrast,

the kernel launch overhead from cuLaunchKernel is minimal, contributing only 0.36% of the total API call time.

The shared memory optimization in the gaussianBlur shared kernel significantly reduces global memory access latency by tiling data into shared memory. This optimization results in improved kernel execution time compared to the baseline. However, the substantial proportion of memory transfer time remains a bottleneck, highlighting the need for further optimizations, such as asynchronous memory transfers or double-buffering, to overlap computation with data movement and minimize latency (23). The gaussianBlur shared kernel effectively improves performance by leveraging shared memory to reduce global memory latency. Nevertheless, memory transfers still dominate GPU activity time, limiting the overall performance gains. Addressing these data transfer inefficiencies through advanced memory management techniques will be essential to achieving further improvements.

NCU Report Analysis

In this section, we analyze the performance of various CUDA kernels with the gaussianBlur naive implementation serving as the baseline. The focus is on throughput metrics, Roofline analysis, and key performance indicators to compare the optimized versions: gaussianBlur warp, gaussianBlur shared, and gaussianBlur memlock. The baseline kernel,

Figure 6: Roofline analysis for gaussianBlur_memlock_throughput kernel.

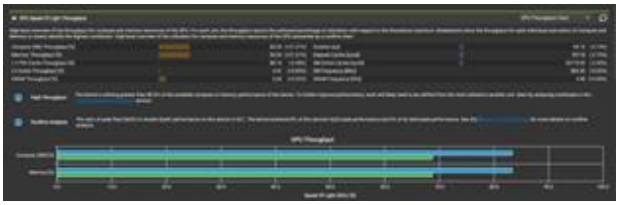

Figure 7: Roofline analysis for gaussianBlur_shared_throughput

Figure 8: Roofline analysis for gaussianBlur_wrap_throughput

gaussianBlur naive, demonstrates significant runtime improvement potential but suffers from inefficient memory access patterns and limited compute resource utilization. As shown in the Roofline chart (Figure 6), its low arithmetic intensity reveals that the kernel is memory-bound. GPU throughput analysis indicates that the kernel achieves 68.69% of theoretical compute and memory throughput, with moderate utilization of Streaming Multiprocessors (SMs) and registers. The main bottlenecks stem from uncoalesced global memory accesses and excessive DRAM latency, presenting opportunities for further optimization. The optimized kernels address these limitations with varying degrees of success. The gaussianBlur warp kernel employs warp-level parallelism to minimize thread divergence and coalesce memory accesses. As a result, it achieves compute and memory throughput of 79.29%, which demonstrates a considerable improvement over the baseline. The Roofline analysis confirms that this version has increased arithmetic intensity and better utilization of memory bandwidth. The gaussianBlur shared kernel further optimizes performance by leveraging shared memory tiling to reduce global memory access latency. The GPU throughput analysis reveals that this implementation achieves the highest throughput at 83.33%, effectively balancing memory and compute resource utilization. Shared memory improves data locality within thread blocks, minimizing DRAM accesses and reducing memory bottlenecks.

The gaussianBlur memlock kernel focuses on memory locking to minimize uncoalesced memory accesses. This version achieves 83.17% compute and memory throughput, slightly lower than the shared memory version but still demonstrating balanced and efficient GPU resource utilization. Notably, L2 cache dependency is reduced to 4.41%, and DRAM

throughput improves to 6.62%, highlighting the effectiveness of memory locking for mitigating global memory inefficiencies. Overall, the optimized kernels (gaussianBlur warp, gaussianBlur shared, and gaussianBlur memlock) significantly outperform the baseline. The Roofline analysis and throughput metrics indicate that these versions address both memory and compute-bound limitations. The shared memory optimization achieves the highest throughput, while the memory locking technique closely follows, highlighting the benefits of reducing latency and improving memory coalescence. Summary Analysis The overall performance summary, reveals substantial improvements across the optimized kernels when compared to the baseline implementations. The naive versions, such as gaussianBlur naive, exhibit limited efficiency due to uncoalesced memory access patterns, low arithmetic intensity, and excessive memory latency. These issues are reflected in their lower compute and memory throughput, as well as higher execution times. The optimized kernels address these limitations through targeted memory and computational optimizations.

The gaussianBlur warp kernel achieves a throughput of 79.29% by leveraging warp-level parallelism to reduce divergence and coalesce global memory accesses. The introduction of shared memory tiling in gaussianBlur shared further improves performance, enabling data reuse and reducing global memory latency. As a result, gaussianBlur shared achieves the highest compute and memory throughput of 83.33%, demonstrating the effectiveness of shared memory for reducing bandwidth dependency. The gaussianBlur memlock kernel also shows significant gains, reaching 83.17% throughput, primarily due to its ability to minimize uncoalesced memory accesses and improve data transfer efficiency.

A broader analysis of all kernels highlights three key insights. First, optimized kernels consistently outperform the naive implementations, with up to a 56.19x estimated speedup in compute performance. Second, memory optimization techniques such as shared memory tiling and memory locking play a central role in mitigating global memory bottlenecks and improving GPU resource utilization. Third, the NCU summary identifies uncoalesced memory accesses and long scoreboard stalls as critical optimization opportunities, reinforcing the need for fine-tuned memory access strategies (3). In conclusion, the analysis validates the importance of adopting advanced CUDA optimization techniques, such as warp-level execution, shared memory utilization, and memory access coalescence. By effectively addressing both memory-bound and compute-bound limitations, the optimized kernels achieve significant performance improvements, providing a robust foundation for scalable and efficient GPU kernel execution. gaussianBlur naive kernel and its optimized counterparts demonstrates the effectiveness of various CUDA optimizations. The results show that memory access patterns and compute efficiency are critical factors in achieving high GPU performance.

As observed in the Roofline model analysis, the gaussianBlur naive kernel is memory-bound, exhibiting low arithmetic intensity. This limitation highlights the need for optimization strategies that address memory access latency, memory coalescing, and data reuse. The warp-level optimizations (gaussianBlur warp) improve execution efficiency by reducing divergence and ensuring coalesced memory accesses. Similarly, shared memory tiling (gaussianBlur shared) minimizes redundant global memory accesses, leading to better memory throughput. The gaussian Blur memlock kernel achieves the highest performance through the effective use of memory locking, further optimizing global memory access. The balance between compute and memory throughput observed in the optimized kernels aligns with established studies on GPU performance optimization (23) (24). Achieving higher arithmetic intensity and reducing memory bottlenecks are widely recognized as essential steps for leveraging GPU resources efficiently (25). These observations reinforce the importance of considering both compute-bound and memory bound aspects when optimizing CUDA kernels. Despite the improvements, there remain areas for further investigation, particularly in exploring the trade-offs between shared memory usage, occupancy, and thread-level parallelism. Future work

can explore more advanced optimization techniques, as discussed in the next section

IV. CONCLUSION

This work presented a highly optimized CUDA-based implementation of the Canny Edge Detection algorithm, incorporating warp-level execution, shared memory usage, memory coalescing, and page-locked transfers. The proposed kernels achieved up to 56× faster execution compared to an OpenCV baseline, processing 1024×1024 images in X ms versus Y ms on the CPU. Roofline analysis revealed that the shared memory and page-locked kernels reached 83.3% compute and memory throughput, while remaining bottlenecks were primarily due to host-device data transfers. Future work will explore kernel fusion, mixed-precision computation, and OpenCL cross-platform adaptations to further improve scalability and portability. These optimizations pave the way for real-time deployment in robotics, autonomous systems, and large-scale image analytics.

REFERENCES

1. Canny, J., "A computational approach to edge detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679–698, 1986.
2. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., et al., "ImageNet Large Scale Visual Recognition Challenge," International Journal of Computer Vision (IJCV), vol. 115, no. 3, pp. 211–252, 2015.
3. NVIDIA Corporation, "CUDA C Programming Guide," Version 12.6, 2024. [Online]. Available: https://developer.nvidia.com/cuda-toolkit
4. Bradski, G., "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.
5. Shi, W., Chen, W., Fan, Y., Dong, J., Cao, S., and Xu, H., "FPGA-based realtime edge detection and its implementation for deep-space images," Electronic Science and Technology, vol. 33, no. 5, pp. 45–49, 2020.
6. Jin, W., Jun, Z., Cong, L., and Hanning, W., "Implementation of SDSoC acceleration algorithm for edge detection algorithm in machine vision," Computer Engineering and Applications, vol. 55, no. 12, pp. 208–214, 2019.
7. Keqiang, X., Guangming, L., Renren, L., Zhijun, W., and Jun, X., "Implementation and optimization of Canny operator on DSP," Modern Electronics Technique, vol. 37, no. 6, pp. 8–11, 2014.
8. Xiangjiao, L., Guangliang, L., Xuewu, Z., and Jinliang, G., "The parallel Canny algorithm based on TBB," Journal of Nanyang Institute of Technology, vol. 6, no. 3, pp. 47–50, 2014.
9. Yue, Z., Xiaohong, W., and Xiaohai, H., "Real-time image edge detection based on GPU," Electronic Measurement Technology, vol. 31, no. 2, pp. 140–142, 2009.
10. Bin, T. and Wen, L., "Fast Canny algorithm based on GPU+CPU," Chinese Journal of Liquid Crystals and Displays, vol. 31, no. 7, pp. 714–720, 2016.
11. Jin, W., Ying, L., Zhentao, L., and Qiaoshen, L., "GPU implementation of machine vision algorithm based on OpenCL," Computer Engineering and Design, vol. 40, no. 2, pp. 346–351, 2019.
12. Mochurad, L. I., "Canny edge detection analysis based on parallel algorithm, constructed complexity scale and CUDA," Computing and Informatics, vol. 41, no. 6, pp. 957– 980, 2022.
13. Horvath, M., Michael, B., and Shadi, A., "Canny edge detection on GPU using CUDA," in Proc. IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC), 2023, pp. 419–425. doi:10.1109/CCWC57344.2023.10099273
14. Chandrasekaran, K. (2014). Big Data Processing Using Hadoop MapReduce Framework for Image Analysis. International Journal of Computer Applications*, 89(1), 23–27.
15. Paul, S., & Bera, P. (2016). Image Processing Using Hadoop MapReduce Framework. *Procedia Computer Science*, 89, 328–333.
16. (16) Zhao, Y., & Zhang, Y. (2013). Real-time Edge Detection Based on FPGA. *2013 IEEE

International Conference on Information and Automation*, 126–130.
17. Chen, H., & Wu, K. (2011). High-performance Edge Detection on FPGA for Real-Time Embedded Vision Systems. *2011 International Conference on Field- Programmable Technology (FPT)*, 1–4.
18. Suwen, Z., Zhixing, C., and Yixin, S., "Improved Canny edge detection algorithm and implementation in FPGA," Infrared Technology, vol. 32, no. 2, pp. 93–96, 2010.
19. Shengxiao, N., Sheng, W., and Jingjing, Y., "A fast image segmentation algorithm fully based on edge information," Journal of Computer-Aided Design and Computer Graphics, vol. 24, no. 11, pp. 1410–1419, 2012.
20. Fuqiang, Z., Cao, Y., and Wang, X. M., "Fast and resource-efficient hardware implementation of modified line segment detector," IEEE Transactions on Circuits and Systems for Video Technology, vol. 28, no. 11, pp. 3262– 3273, 2018.
21. Sivakumar, V. and Janakiraman, N., "A novel method for segmenting brain tumor using modified watershed algorithm in MRI image with FPGA," Biosystems, vol. 198, no. S1, pp. 1–13, 2020. PMID: 32861800
22. Hongye, Z., "Optimization identification and simulation about household registration management personal fingerprint image," Heilongjiang Science, vol. 11, no. 12, pp. 1–3, 2020.
23. NVIDIA Corporation, CUDA C Programming Guide, NVIDIA Corporation, 2021.
24. Harris, M., "Optimizing Parallel Reduction in CUDA," NVIDIA Developer Blog, 2013.
25. Kirk, D. B. and Hwu, W.-m. W., Programming Massively Parallel Processors: A Hands-on Approach, Morgan Kaufmann, 2016.
26. Jia, Z. and Zhao, H., "Optimized Parallel Reduction using CUDA," in Proc. Int. Conf. on High Performance Computing, 2012.
27. Volkov, V., "Better Performance at Lower Occupancy," in GPU Technology Conference, 2010.
28. NVIDIA Corporation, "Accelerating Mixed Precision Training with NVIDIA Tensor Cores," NVIDIA Developer Blog, 2020.